a software team to accommodate changes late in a software project without dramatic cost and time impact. You've already learned that the agile process encompasses incremental delivery. When incremental delivery is coupled with other agile practices such as continuous unit testing and pair programming (discussed briefly in Section 3.5.1 and in more detail in Chapter 20), the cost of making a change is attenuated. Although debate about the degree to which the cost curve flattens is ongoing, there is evidence [Coc01a] to suggest that a significant reduction in the cost of change can be achieved.

3.3 What Is an Agile Process?

Any agile software process is characterized in a manner that addresses a number of key assumptions [Fow02] about the majority of software projects:

1. It is difficult to predict in advance which software requirements will persist and which will change. It is equally difficult to predict how customer priorities will change as the project proceeds.
2. For many types of software, design and construction are interleaved. That is, both activities should be performed in tandem so that design models are proven as they are created. It is difficult to predict how much design is necessary before construction is used to prove the design.
3. Analysis, design, construction, and testing are not as predictable (from a planning point of view) as we might like.

Given these three assumptions, an important question arises: How do we create a process that can manage *unpredictability*? The answer, as we have already noted, lies in process adaptability (to rapidly changing project and technical conditions). An agile process, therefore, must be *adaptable.*

But continual adaptation without forward progress accomplishes little. Therefore, an agile software process must adapt *incrementally.* To accomplish incremental adaptation, an agile team requires customer feedback (so that the appropriate adaptations can be made). An effective catalyst for customer feedback is an operational prototype or a portion of an operational system. Hence, an *incremental development strategy* should be instituted. *Software increments* (executable prototypes or portions of an operational system) must be delivered in short time periods so that adaptation keeps pace with change (unpredictability). This iterative approach enables the customer to evaluate the software increment regularly, provide necessary feedback to the software team, and influence the process adaptations that are made to accommodate the feedback.

3.3.1 Agility Principles

The Agile Alliance [Agi17][2] defines 12 principles for those software organizations that want to achieve agility. These principles are summarized in the paragraphs that follow.

Customer satisfaction is achieved by providing value through software that is delivered to the customer as rapidly as possible. To achieve this, agile developers

2 The Agile Alliance home page contains much useful information: https://www.agilealliance.org/.

recognize that requirements will change. They deliver software increments frequently and work together with all stakeholders so that feedback on their deliveries is rapid and meaningful.

An agile team is populated by motivated individuals, who communicate face-to face and work in an environment that is conducive to high quality software development. The team follows a process that encourages technical excellence and good design, emphasizing simplicity—"the art of maximized the amount of work not done" [Agi17]. Working software that meets customer needs is their primary goal, and the pace and direction of the team's work must be "sustainable," enabling them to work effectively for long periods of time.

An agile team is a "self-organizing team"—one that can develop well-structured architectures that lead to solid designs and customer satisfaction. Part of the team culture is to consider its work introspectively, always with the intent of improving the manner in which it addresses its primary goal.

Not every agile process model applies characteristics described in this section with equal weight, and some models choose to ignore (or at least downplay) the importance of one or more agile principles. However, these principles define an *agile spirit* that is maintained in each of the process models presented in this chapter.

3.3.2 The Politics of Agile Development

There is considerable debate (sometimes strident) about the benefits and applicability of agile software development as opposed to more conventional software engineering processes. Jim Highsmith [Hig02a] (facetiously) states the extremes when he characterizes the feeling of the pro-agility camp ("agilists"): "Traditional methodologists are a bunch of stick-in-the-muds who'd rather produce flawless documentation than a working system that meets business needs." As a counterpoint, he states (again, facetiously) the position of the traditional software engineering camp: "Lightweight, er, 'agile' methodologists are a bunch of glorified hackers who are going to be in for a heck of a surprise when they try to scale up their toys into enterprise-wide software."

Like all software technology arguments, this methodology debate risks degenerating into a religious war. If warfare breaks out, rational thought disappears and beliefs rather than facts guide decision making.

No one is against agility. The real question is: What is the best way to achieve it? Keep in mind that working software is important, but don't forget that it must also exhibit a variety of quality attributes including reliability, usability, and maintainability. How do you build software that meets customers' needs today and exhibits the quality characteristics that will enable it to be extended and scaled to meet customers' needs over the long term?

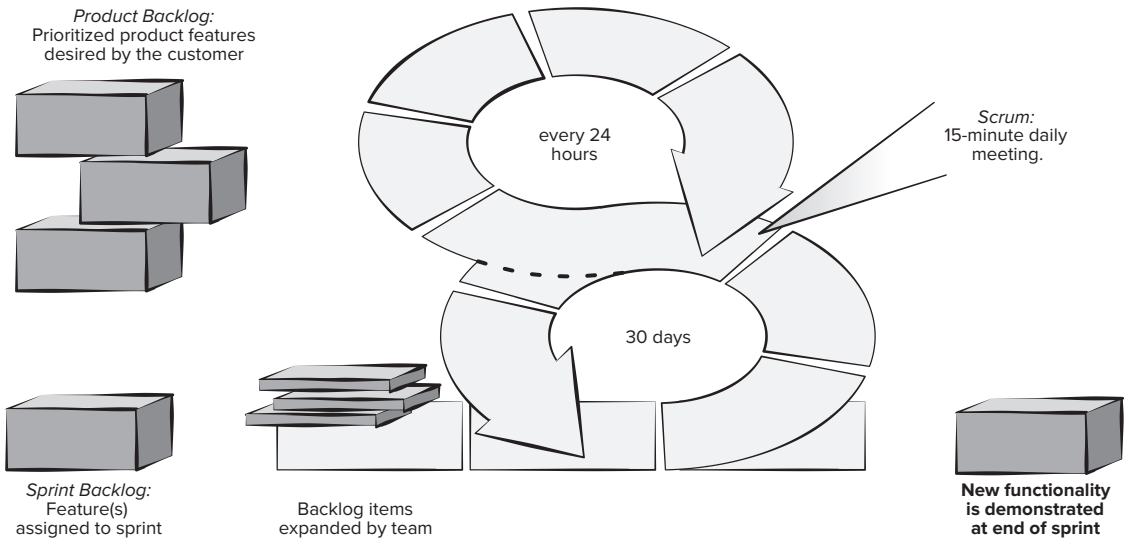
There are no absolute answers to either of these questions. Even within the agile school itself, there are many proposed framework models (Sections 3.4 and 3.5), each with a subtly different approach to the agility problem. Within each model there is a set of "ideas" (agilists are loath to call them "work tasks") that represent a significant departure from traditional software engineering. And yet, many agile concepts are simply adaptations of good software engineering concepts. The bottom line is there is much that can be gained by considering the best of both schools and virtually nothing to be gained by denigrating either approach.

3.4 SCRUM

Scrum (the name is derived from an activity that occurs during a rugby match)[3] is a very popular agile software development method that was conceived by Jeff Sutherland and his development team in the early 1990s. Further development on the Scrum methods was performed by Schwaber and Beedle [Sch01b].

Scrum principles are consistent with the agile manifesto and are used to guide development activities within a process that incorporates the following framework activities: requirements, analysis, design, evolution, and delivery. Within each framework activity, work tasks take place in a relatively short time-boxed[4] period called a *sprint*. The work conducted within a sprint (the number of sprints required for each framework activity will vary depending on size of the product and its complexity) is adapted to the problem at hand and is defined and often modified in real time by the Scrum team. The overall flow of the Scrum process is illustrated in Figure 3.2. Much of our description of the Scrum framework appears in Fowler and Sutherland [Fow16].[5]

FIGURE 3.2 **Scrum process flow**



3 A group of players forms around the ball, and the teammates work together (sometimes violently!) to move the ball downfield.

4 A *time-box* is a project management term (see Part Four of this book) that indicates a period of time that has been allocated to accomplish some task.

5 The Scrum Guide is available at: https://www.Scrum.org/resources/what-is-Scrum.

SAFEHOME



Considering Agile Software Development

The scene: Doug Miller's office.

The players: Doug Miller, software engineering manager; Jamie Lazar, software team member; Vinod Raman, software team member.

The conversation: (A knock on the door, Jamie and Vinod enter Doug's office.)

Jamie: Doug, you got a minute?

Doug: Sure Jamie, what's up?

Jamie: We've been thinking about our process discussion yesterday . . . you know, what process we're going to choose for this new *SafeHome* project.

Doug: And?

Vinod: I was talking to a friend at another company, and he was telling me about Scrum. It's an agile process model . . . heard of it?

Doug: Yeah, some good, some bad.

Jamie: Well, it sounds pretty good to us. Lets you develop software really fast, uses something called sprints to deliver software increments when the team decides the product is done . . . it's pretty cool, I think.

Doug: It does have a lot of really good ideas. I like the sprint concept, the emphasis on early test case creation, and the idea that the process owner should be part of the team.

Jamie: Huh? You mean that marketing will work on the project team with us?

Doug (nodding): They're stakeholders but not really the product owner. That would be Marg.

Jamie: Good. She will filter the changes marketing will want to send every 5 minutes.

Vinod: Even so, my friend said that there are ways to "embrace" changes during an agile project.

Doug: So you guys think we should use Scrum?

Jamie: It's definitely worth considering.

Doug: I agree. And even if we choose an incremental model as our approach, there's no reason why we can't incorporate much of what Scrum has to offer.

Vinod: Doug, before you said "some good, some bad." What was the bad?

Doug: The thing I don't like is the way Scrum downplays analysis and design . . . sort of says that writing code is where the action is . . .

(The team members look at one another and smile.)

Doug: So you agree with the Scrum approach?

Jamie (speaking for both): It can be adapted to fit our needs. Besides, writing code is what we do, Boss!

Doug (laughing): True, but I'd like to see you spend a little less time coding and then recoding and a little more time analyzing what needs to be done and designing a solution that works.

Vinod: Maybe we can have it both ways, agility with a little discipline.

Doug: I think we can, Vinod. In fact, I'm sure of it.

3.4.1 Scrum Teams and Artifacts

The Scrum team is a self-organizing interdisciplinary team consisting of a *product owner*, a *Scrum master,* and a small (three to six people) *development team.* The principle Scrum artifacts are the *product backlog*, the *sprint backlog*, and the code *increment*. Development proceeds by breaking the project into a series of incremental prototype development periods 2 to 4 weeks in length called *sprints*.

The product backlog is a prioritized list of product requirements or features that provide business value for the customer. Items can be added to the backlog at any time with the approval of the product owner and the consent of the development team. The product owner orders the items in the product backlog to meet the most important goals of all stakeholders. The product backlog is never complete while the product is evolving to meet stakeholder needs. The product owner is the only person who decides whether to end a sprint prematurely or extend the sprint if the increment is not accepted.

The sprint backlog is the subset of product backlog items selected by the product team to be completed as the code increment during the current active sprint. The increment is the union of all product backlog items completed in previous sprints and all backlog items to be completed in the current sprints. The development team creates a plan for delivering a software increment containing the selected features intended to meet an important goal as negotiated with the product owner in the current sprint. Most sprints are time-boxed to be completed in 3 to 4 weeks. How the development team completes the increment is left up to the team to decide. The development team also decides when the increment is done and ready to demonstrate to the product owner. No new features can be added to the sprint backlog unless the sprint is cancelled and restarted.

The Scrum master serves as facilitator to all members of the Scrum team. She runs the daily Scrum meeting and is responsible for removing obstacles identified by team members during the meeting. She coaches the development team members to help each other complete sprint tasks when they have time available. She helps the product owner find techniques for managing the product backlog items and helps ensure that backlog items are stated in clear and concise terms.

3.4.2 Sprint Planning Meeting

Prior to beginning, any development team works with the product owner and all other stakeholders to develop the items in the product backlog. Techniques for gathering these requirements are discussed in Chapter 7. The product owner and the development team rank the items in the product backlog by the importance of the owner's business needs and the complexity of the software engineering tasks (programming and testing) required to complete each of them. Sometimes this results in the identification of missing features needed to deliver the required functionality to the end users.

Prior to starting each sprint, the product owner states her development goal for the increment to be completed in the upcoming sprint. The Scrum master and the development team select the items to move from to the sprint backlog. The development team determines what can be delivered in the increment within the constraints of the time-box allocated for the sprint and, with the Scrum master, what work will be needed to deliver the increment. The development team decides which roles are needed and how they will need to be filled.

3.4.3 Daily Scrum Meeting

The daily Scrum meeting is a 15-minute event scheduled at the start of each workday to allow team members to synchronize their activities and make plans for the next

24 hours. The Scrum master and the development team always attend the daily Scrum. Some teams allow the product owner to attend occasionally.

Three key questions are asked and answered by all team members:

- What did you do since the last team meeting?
- What obstacles are you encountering?
- What do you plan to accomplish by the next team meeting?

The Scrum master leads the meeting and assesses the responses from each person. The Scrum meeting helps the team to uncover potential problems as early as possible. It is the Scrum master's task to clear obstacles presented before the next Scrum meeting if possible. These are not problem-solving meetings, those occur off-line and only involve the affected parties. Also, these daily meetings lead to "knowledge socialization" [Bee99] and thereby promote a self-organizing team structure.

Some teams use these meetings to declare sprint backlog items complete or done. When the team considers all sprint backlog items complete, the team may decide to schedule a demo and review of the completed increment with the product owner.

3.4.4 Sprint Review Meeting

The sprint review occurs at the end of the sprint when the development team has judged the increment complete. The sprint review is often time-boxed as a 4-hour meeting for a 4-week sprint. The Scrum master, the development team, the product owner, and selected stakeholders attend this review. The primary activity is a *demo* of the software increment completed during the sprint. It is important to note that the demo may not contain all planned functionality, but rather those functions that were to be delivered within the time-box defined for the sprint.

The product owner may accept the increment as complete or not. If it is not accepted, the product owner and the stakeholders provide feedback to allow a new round of sprint planning to take place. This is the time when new features may be added or removed from the product backlog. The new features may affect the nature of the increment developed in the next sprint.

3.4.5 Sprint Retrospective

Ideally, before beginning another sprint planning meeting, the Scrum master will schedule a 3-hour meeting (for a 4-week sprint) with the development team called a *sprint retrospective*. During this meeting the team discusses:

- What went well in the sprint
- What could be improved
- What the team will commit to improving in the next sprint

The Scrum master leads the meeting and encourages the team to improve its development practices to become more effective for the next sprint. The team plans ways to improve product quality by adapting its definition of "done." At the end of this meeting, the team should have a good idea about the improvements needed in the next sprint and be ready to plan the increment at the next sprint planning meeting.